The Evaluation of Reliability Based on the Software Architecture in Neural Networks

Maryam Harirforoush
Islamic Azad University - South Tehran Branch
maryam7501@ yahoo.com

Dr. Mirali Seyyedi
Islamic Azad University - South Tehran Branch
maseyyedi @gmail.com

Dr. Nooraldeen Mirzaee
Islamic Azad University - South Tehran Branch
mirza683@yahoo.com

Abstract

Software reliability is one of the key quality attributes. To avoid reworks after developing software, mentioned attribute of software must be evaluated correctly. Therefore, Estimation and prediction of reliability during system development are very important. In this article a method to predict the failure probability in whole system by the neural networks has been presented. The model of this neural network varies based on the architectural style of software. An evaluation model has been implemented for one architectural style of software as a case study.

1. Introduction

There is more need for software reliability due to making the performance of computer operation more complicated. Nowadays, reliability in software engineering is allocated the most parts of theoretical and practical discussions. There are different methods for evaluation of software architecture each of which focuses on some qualitative features of architecture such as reliability that are extensively scenario-based. The methods [3,6] include: Software Architecture Analysis Method (SAAM), SAAM Founded on Complex Scenario (SAAMCS), Extending SAAM by Integration in the Domain (ESAAMI), Software Architecture Analysis for Evaluation and Reusability (SAAMER), Architecture Trade-off Analysis Method [8] (ATAM), Cost-Benefit Analysis Method (CBAM), Architecture Level Modifiability Analysis [7] (ALMA), Family-Architecture Assessment Method (FAAM) and etc. Of the deficiencies of these methods are that since they are scenario-based, there is not a definite quantitative measurement of analyzed architectural features. Therefore, they can't assess the reliability quantitatively. Quantitative method based on architecture is Cheung method [12] that it computes the reliability of whole system in different styles of architecture through obtaining the reliability of each component, having high costs due to the testing of each component to reach the reliability .In this paper, neural networks models has been applied for software reliability estimation based on architecture. This idea has been implemented for one of software architecture styles (fault tolerance style) as sample model. In this sample, multilayer perceptron (MLP) neural network has been used due to its high similarity with above-mentioned style for prediction of failure probability of whole system. In this model, neurons are estimator of reliability of architectural components. The inputs in this neural network are the failure probability of each component .So, training the neural network can predict the failure probability of whole system.

There will be offered first the comprehensive definition of software reliability in section 2. In section 3, software architecture and its components are defined. Section 4 represents software architecture styles. The methods for software architecture evaluation, scenario-based will be determined in section 5. Section 6 predicts the failure probability of whole system by neural networks. Results have been shown in section 7 and finally in section 8, the conclusion and future works are offered.

2. Software reliability

Reliability is defined as the succeeding probability of a system. Mathematically, reliability R (t) is a probability that a system will be successful in the interval from time 0 to time t, whereas T is a random variable showing the time-to-failure or failure time, represented as following [4]:

$$R(t) = p(T > t), t \geq 0 \qquad (1)$$

978-0-7695-3372-8/08 $25.00 © 2008 IEEE
DOI 10.1109/ICSEA.2008.56

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 07,2026 at 18:48:53 UTC from IEEE Xplore. Restrictions apply.

If the random variable of failure time T has a density function f (t), then R (t) equals to:

$$R(t) = \int_{t}^{\infty} f(s)ds, f(t) = \frac{-d}{dt}[R(t)] \quad (2)$$

The density function can be mathematically described in terms of T as follows:

$$\lim_{\Delta t \to 0} p(t < T \leq t + \Delta t) \quad (3)$$

The expression can be interpreted as the probability that the failure time T will occur between the operation time t and the next interval of operation t+Δt.

3. Software architecture and its components

Many persons have described the different definitions during various years, some of which are referred here.

Perry and Wolf defined the software architecture in 1992 as: [9] "It is as a set of architectural elements that have a particular form. Architectural elements include processing, data, and connecting elements."

Garlan and Shaw proposed the following definition in 1994 as: [9] "The software architecture is a collection of components and connectors together with a description of the interaction between these components."

Bass, Clements and Kazman offered the following definition in 1998 as: [1] "The software architecture of a program or computing system is the structure or structures of the system, which comprise software elements, the externally visible properties of those elements, and the relationship among them."

Finally, IEEE defined the software architecture in 2000 as: "It is the primarily organizing a system insisting of the components, their interrelations and their relations with system environment and the principles governing on the designing and its development."

The attributes of software architecture include: components, connectors and interfaces. Components [5] are used as the primary blocks and accounting entities in construction of a system, relating with outer area through one or more ports. The port can be an interface, a common variable or the other mechanisms. Connectors define the relations between variables and describe the principles governing on them. A connector can connect the ports of two or more components to each other. When the connector links two components, the component defines an interface, that the interfaces are related only to one component.

4. Software architecture styles

There are different definitions on the software architecture styles. Shaw declared in 1996 that architecture style defines in terms of a pattern of interactions among typed components. Albin described in 2003 that an architecture style is a kind of ultramodel to determine the collection of attributes and their interrelations for assigning a system based on the style.

There are different architecture styles. These styles include: data-centered style, data-flow style, virtual machine style, call-and-return style, independent component style, heterogeneous styles and etc. We are to analyze now one of sample styles, named as Fault Tolerance. It consists of the primary components and a set of backup components that can be implemented with different algorithms or in various structures according to the primary components.

The components of this architecture style, including the primary and the backups, are placed in parallel so that when a primary component is damaged or in failure, the system, by replacing of backup components, can offer the service. As Figure 1 (a) shows architecture view, the components of pointed lines from C_3 to C_{K-3} are the backup components for primary component C_2.

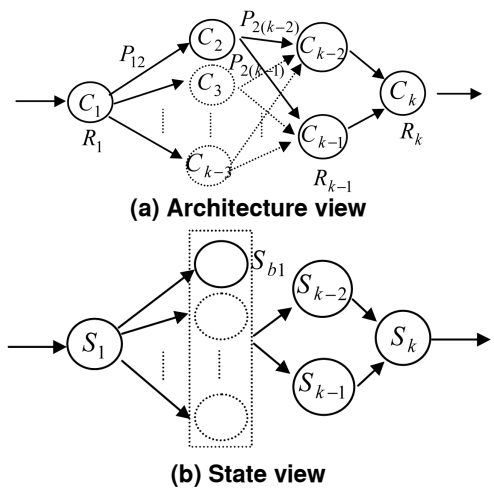

Figure 1. Fault tolerance [11].

In this style, whenever state S_2 fails, state S_3 will be replaced and cause to improve the reliability, respectively, if both states S_2 and S_3 fail, S_4 will be backup component, to S_{k-3}.

5. Scenario-based software architecture evaluation methods

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 07,2026 at 18:48:53 UTC from IEEE Xplore. Restrictions apply.

Among the questionable techniques of software architecture evaluation, scenario-based evaluation techniques are more important than others due to the evaluating one quality feature through making a scenario profile. One of these methods is Software Architecture Analysis Method (SAAM). It is the first method of software architecture evaluation, scenario-based, used for evaluating some architecture qualitative attributes such as modifiability, portability, extensibility and reliability. It is applied for the comparison of different architectures. To assess the scenario, SAAM focuses on the group of architectural elements on which the scenario is effective.

One benefit in mentioned method is that the relation between stakeholder's system increases, but this method is completely analytical and exact calculation model according to quantitative criteria is not represented by this method.

As shown before, the methods are analytical and they do not represent an exact quantitative evaluation. Cheung [12] offered a method for measuring the reliability of whole system, analyzed in Fault Tolerance. In his method, the state diagram is used; it is a directed graph, each node of which, S_i, shows a state and the transition from state S_i to S_j is represented by directed edge (S_i, S_j). R_i shows the state reliability of S_i and P_{ij} is the probability of transition from state S_i to S_j that transition matrix M is defined based on the state diagram as following:

$$M = \begin{array}{c} \\ S_1 \\ S_2 \\ \vdots \\ S_i \\ \vdots \\ S_{n-1} \\ S_n \end{array} \begin{array}{c} \begin{array}{ccccccc} S_1 & S_2 & .. & S_i & .. & S_{n-1} & S_n \end{array} \\ \begin{bmatrix} 0 & R_1P_{12} & .. & R_1P_{1i} & .. & R_1P_{1(n-1)} & R_1P_{1n} \\ R_2P_{21} & 0 & .. & R_2P_{2i} & .. & R_2P_{2(n-1)} & R_2P_{2n} \\ \vdots & \vdots & .. & \vdots & .. & \vdots & \vdots \\ R_iP_{i1} & R_iP_{i1} & .. & 0 & .. & R_iP_{i(n-1)} & R_iP_{in} \\ \vdots & \vdots & .. & \vdots & .. & \vdots & \vdots \\ R_{n-1}P_{(n-1)1} & R_{n-1}P_{(n-1)2} & .. & R_{n-1}P_{(n-1)i} & .. & 0 & R_{n-1}P_{(n-1)n} \\ R_nP_{n1} & R_nP_{n2} & .. & R_nP_{ni} & .. & R_nP_{n(n-1)} & 0 \end{bmatrix} \end{array} \quad (4)$$

The value of the entry M (i,j) which can be computed as $R_i \times P_{ij}$ that indicates the successful arrival at state S_j from S_i. Let S= {S_1, S_2, …,S_n} be the set of states in the state diagram where S_1 is the initial state and S_n is the final state. M^K (i,j) represents the probability of reaching state S_j from S_i through k transitions. Therefore, the reliability R beginning from S_i to S_j with total k transitions is represented as follows:

$$R = M^K(i,j) \times R_j \quad (5)$$

Matrix T is considered as following which I represents identity matrix of size n×n:

$$T = I + M + M^2 + \ldots = \sum_{K=0}^{\infty} M^K \approx \frac{I}{I-M} = (I-M)^{-1} \quad (6)$$

Then the overall system reliability of can be computed as follows:

$$R = T(1,n) \times R_n$$
$$T(1,n) = (-1)^{n+1} \frac{|E|}{|I-M|} \quad (7)$$

Whereas |I-M| is the determinant of matrix (I-M) and |E| is the determinant of the remaining matrix excluding the nth row and first column of the matrix (I-M).If system in large scale has a set of initial states such as I={S_{i1},S_{i2} , …,S_{im}} and a set of final states such as F={S_{f1}, S_{f2}, …, S_{fn}}, then the problem of multiple initial and final states to one initial state and one final state is converted to a super-initial (S^I) and a super-final (S^F) state to the state diagram(Figure 2[12]).

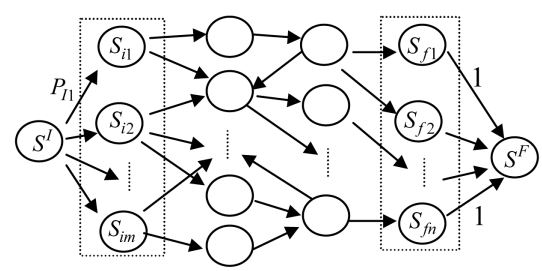

Figure 2. One super-initial and one super-final state diagram [11].

So, in state diagram has added a super-initial state S^I and a directed edge (S^I, S_{ij}) with its transition probability P_{Ij} , observed from the operational profile, for each j = 1, 2, …., m and also a super-final state S^F and a directed edge (S_{fj}, S^F) with transition probability 1 for each j = 1, 2,…. , *n*. Here R_i, the reliability of C_i and P_{ij}, is the probability of transition from component C_i to C_j.

According to the state view of Figure 1(b), the components C_2 to C_{k-3} are in the state of S_{b1} that is a member of states collection of S_b.

Therefore, reliability of reaching the states S_{k-1} and S_{k-2} from S_1 can be computed in following way:

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 07,2026 at 18:48:53 UTC from IEEE Xplore. Restrictions apply.

$$R_{b1} = R_2 + \sum_{n=3}^{k-3}((\prod_{m=2}^{n-1}(1-R_m))R_n) \quad (8)$$

Thus in fault tolerance architecture with k components, the transition matrix can be constructed as follows:

$$\begin{cases} M(i,j) = R_i P_{ij}, S_i \notin S_b \\ M(i,j) = R_{a1} + \sum_{q=a2}^{ar}((\prod_{m=a1}^{q-1}(1-R_m))R_n) \\ , S_i \in S_b \quad and \quad S_i \quad includes \quad C_{a1} \; to \quad C_{ar} \\ M(i,j) = 0, S_i \; can't \quad reach \quad S_j \\ , for \; 1 \le i, j \le |S| \quad and \; 1 \le a_r \le K \end{cases} \quad (9)$$

6. Prediction of failure probability of whole system through neural networks

At first, the structure of neural network MLP [13] should be analyzed. The network contains an input layer, an output layer and one or more hidden layers, each of which has formed by several neurons. The relationship needs to be present between all neurons of one layer and all neurons of next layer that the relationship is determined by weight coefficient. Any neuron has a bias (threshold) and an activity function (output limiter). For example, consider the following multilayer perceptron with one hidden layer and one output layer (Figure 3[2]).

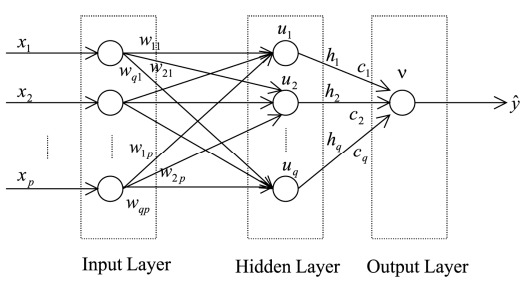

Figure 3. Multilayer perceptron with one hidden layer and one output layer [2].

The input layer receives the inputs x_1, x_2, …,x_P as the inputs. Indeed, each neuron of the input layer receives an input signal and passes it as output without any computation in the neuron. In this method:

$$u_i = \sum_{j=1}^{p} w_{ij} x_j, h_i = \varphi(u_i), i = 1,2,...,q \quad (10)$$

$$\nu = \sum_{l=1}^{q} c_l h_l, \hat{y} = \varphi(\nu) \quad (11)$$

The output of output layer is shown with $\hat{y}$, h_i is the output of hidden layer and w_{ij} is the connected weight coefficient between neurons i and j. The activation function of each neuron is supposed as following, called sigmoid function:

$$\varphi(\nu) = \frac{1}{1+\exp(-\nu)} \quad (12)$$

In Fault Tolerance, like the Figure 4, the neural network MLP [10] is used to predict the failure probability of whole system. Therefore, input X_1 is the failure probability of component i and output $\hat{y}$ is the failure probability of whole system. In this style, two hidden layers are used and the number of layers is certainly extendable. First in the input layer, those inputs showing the failure probability enter into system. Each component of a probability causes in a failure of next component and this failure is distributed in all components. The connected weight coefficient between two components equals to the efficiency level of component in failure distribution.

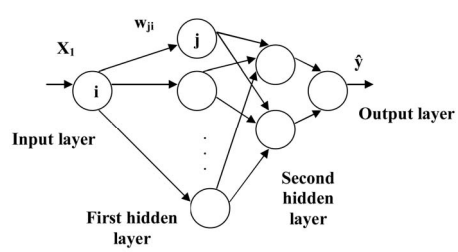

Figure 4. Fault tolerance style according to neural network MLP.

The neural network needs training. The available experiences of previous softwares can be used in network training. To train the network, the algorithm of EBP[1] is used, and the number of training steps is based upon the number of training data. In first step of algorithm, the inputs, related weight coefficient, biases and the output must be determined and the changes of weight coefficients and bias are computed, so that the network can be trained in the next steps by giving the inputs and outputs. Based on the following formulas, there can compute the changes of weight coefficients and outputs of each neuron:

[1]- Error-Back-Propagation

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 07,2026 at 18:48:53 UTC from IEEE Xplore. Restrictions apply.

$$
\begin{aligned}
a_{pj} &= \sum_i w_{ji} o_{pi} + u_j \\
o_{pj} &= \frac{1}{1+e^{-a_{pj}}} \\
\Delta_p w_{ji} &= \beta \partial_{pj} o_{pi}, 0 \prec \beta \le 1 \\
\partial_{pj} &= (t_{pj} - o_{pj}) o_{pj} (1 - o_{pj}) \\
\partial_{pj} &= (\sum_{k=1}^{K} \partial_{pk} w_{kj}) o_{pj} (1 - o_{pj})
\end{aligned}
\tag{13}
$$

Whereas β is learning rate, u_j the level of bias (threshold) of J-th neuron, t_{pj} the desired value (goal) for the output of j-th neuron in step p, a_{pj} the level of j-th neuron activity in p step, o_{pj} the output of j-th neuron in p step, $\Delta_p w_{ji}$ the Level of change in weight coefficient w_{ji} in p step and ∂_{pj} respectively as the Level of error in output layer and as the Level of error in hidden layer. After completing the training network, network can be tested.

7. Checking of the results

125 samples have been used for implementation which they are as data sets that 100 samples are used for training data and 25 samples are used for testing data. Each sample includes 11 inputs and 1 output. These 11 inputs indicate failure probability of each component. Two layers have been used in this model that first layer and second layer have 40 neurons and 1 neuron, respectively. Diagram of training the neural network has been represented in Figure 5.

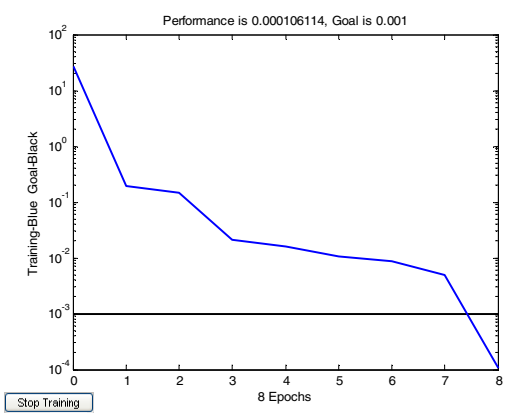

Figure 5. Diagram of training the neural network.

Also comparative diagram of output of neural network and actual output of the model for training data and testing data have been shown in Figure 6 and Figure 7, respectively. Mean square error is 0.0075524.

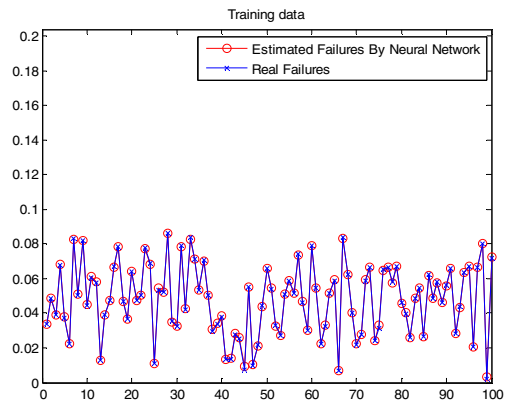

Figure 6. Comparative diagram of output of neural network and actual output of the model for training data.

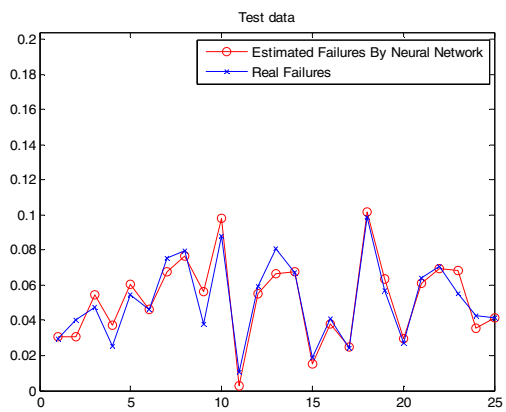

Figure 7. Comparative diagram of output of neural network and actual output of the model for testing data.

8. Conclusion and future works

In this article, the neural network MLP was used in one of the software architecture styles (Fault Tolerance) to predict the failure probability of whole system, and each component of a probability causes the failure in next component. The inputs of this network are the failure probability of the component that the failure probability of whole system can be predicted by training the neural network. Some advantages of this method are predictability, cost and time saving, quantifying and formalizing method. In future, it can be extended into other software architecture styles.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 07,2026 at 18:48:53 UTC from IEEE Xplore. Restrictions apply.

References

[1] F. Martensson, *Software Architecture Quality Evaluation-Approaches in an Industrial Context*, Bleking Institute of Technology, Sweden, 2006.

[2] K. Y. Cai, L. Cai, W. D. Wang, Z. Y. Yu, and D. Zhang, "On the Neural Network Approach in Software Reliability Modeling", *The Journal of Systems and Software*, Elsevier, New York, 1 August 2000, pp. 48-49.

[3] L. Dobrica, and E. Niemela, "A Survey on Software Architecture Analysis Methods", *IEEE Transactions on Software Engineering*, IEEE, Bucharest, July 2002, pp. 641-651.

[4] M. R. T. Lyu, "Software Reliability Theory", *Encyclopedia of Software Engineering*, John Wiley & Sons, Hong Kong, 15 January 2002, pp. 9-10 .

[5] M. S. Mahmoud, and M. M. Shaban, "A Framework of Architectural Styles for Distributed Business Information Systems", *IJICIS*, Egypt, July 2005, pp. 69-70.

[6] M. T. Ionita, D. K. Hammer, and H. Obbink, "Scenario-Based Software Architecture Evaluation Methods: An Overview", *Technical University*, Eindhoven, Florida, May 2002, pp. 1-10.

[7] P. Bengtsson, N. Lassing, J. Bosch, and H. V. Vliet, "Architecture-Level Modifiability Analysis (ALMA)", *The Journal of Systems and Software*, Elsevier, New York, 3 June 2002, pp. 129-146.

[8] R. Kazman, M. Klein, and P. Clements, "ATAM: Method for Architecture Evaluation", *CMU/SEI-2000-TR-004*, SEI, Pittsburgh, August 2000, pp. 2-63.

[9] R. T. Fielding, "Architecture Styles & the Design of Network-Based Software Architecture", phD Thesis, University of California, Irvine, 2000, pp. 3-6.

[10] W. H. Delashmit, and M. T. Manry, "Recent Developments in Multilayer Perceptron Neural Networks", *Proceedings of 7th Annual Memphis Area Engineering and Science Conference*, MAESC, USA, 2005, pp. 1-3.

[11] W. L. Wang, D. Pan, and M. H. Chen, "Architecture-Based Software Reliability Modeling", *The Journal of Systems and Software*, Elsevier, Albany, 2006, pp. 7-12.

[12] W. L. Wang, Y. Wu, and M. Chen, "An Architecture-Based Software Reliability Model", *Proceeding of the 1999 Pacific Rim International Symposium on Dependable Computing*, IEEE, Hong Kong, 1999, pp.1-4.

[13] Y. S. Su, and C. Y. Huang, "Neural-Network-Based Approaches for Software Reliability Estimation Using Dynamic Weighted Combinational Models", *The Journal of Systems and Software*, Elsevier, Taiwan, 4 August 2006, pp. 608-609.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 07,2026 at 18:48:53 UTC from IEEE Xplore. Restrictions apply.